Specialist review agents for AI-assisted development workflows

Six specialist agents—architecture, security review, TDD validation, pre-commit explanation, security diagnosis, and performance diagnosis—should each operate as isolated Claude Code subagents with XML-structured prompts under 2,500 tokens, terminal-tool output contracts enforcing structured JSON, and phase-gated invocation tied to your 8-phase methodology. The evidence base from Stripe (1,300+ AI-authored PRs/week), The Hans India Ellipsis (27 months in production), tribe Shopify (ICML 2025), Shopify Engineering and Anthropic's own subagent architecture converges on a single design principle: many small specialists with restricted tool access outperform one mega-agent every time. tribe This report delivers prompt structures, invocation triggers, input/output contracts, failure modes, and MCP tooling recommendations for each agent, grounded in 2024–2026 production evidence.

The tech stack—FastAPI, SQLAlchemy 2.0, PostgreSQL with pgvector, Redis, Celery, LangGraph, Linear, Figma, GitHub—has mature MCP server coverage across every integration. The critical gap is hallucinated dependency detection, where **19.7% of LLM-recommended packages are fabricated** Trendmicro (USENIX Security 2025), diffray but MCP-compatible tools now exist to close it.

The subagent architecture that actually works in production

Claude Code's subagent system (v2.1.68+) provides the correct primitive for specialist review agents. github Each subagent receives only its custom system prompt plus basic environment details—not the full Claude Code system prompt. Claude This isolation prevents context pollution, the single largest failure mode in production agent systems.

The canonical file format lives at `.claude/agents/<agent-name>.md`:

```yaml
---
name: architecture-reviewer
description: Evaluates plans against SOLID, coupling risks, and schema separation
tools: Read, GlobTool, GrepTool
model: sonnet
---
<system prompt body>
```

Three design decisions matter more than anything else for these agents:

First, use XML tags in prompt bodies. Anthropic's documentation states XML tags reduce misinterpretation for complex prompts mixing instructions, context, and output format. Claude API Docs Every production agent surveyed (Archbot, Ellipsis, GitHub Copilot custom

agents) uses structured sections rather than prose instructions. The recommended structure is `<role>`, `<instructions>`, `<output_format>`, `<constraints>`, `<if_unsure>`.

Second, enforce structured output via a terminal tool pattern. Archbot's production system requires the model to call a `submit_review` function with typed fields (Platform Toolsmith) (severity enum, file/line citation, evidence string, suggestion) rather than producing freeform Markdown. This makes it harder to hand-wave. (dev) Ellipsis independently converged on the same pattern. When the model must populate fields rather than write prose, false positive rates drop measurably because the model is forced to cite evidence. (dev)

Third, keep prompts between 1,500 and 2,500 tokens. GitHub Copilot hard-caps custom instructions at 4,000 characters. (GitHub) Cursor caps at 6,000 characters per rule file. (DeployHQ) Claude Code's built-in verification specialist runs at ~2,453 tokens. (GitHub) Anthropic's "progressive disclosure" principle—show just enough, then reveal more on demand—is the right mental model. (Lee Hanchung) If your agent prompt exceeds 3,000 tokens, you are almost certainly mixing concerns.

Anti-pattern: the growing system prompt. Shopify's ICML 2025 paper documents "Death by a Thousand Instructions"—as their tool count grew past 20, system prompt bloat degraded performance. Past 50 tools, the model couldn't reliably distinguish between tools doing similar things. Their fix: JIT (Just-in-Time) instructions delivered alongside tool responses instead of front-loaded in the system prompt. (Shopify Engineering) (shopify) For review agents specifically, this means embedding context in the input payload, not the system prompt.

Agent 1: Architecture review at the planning gate

Invocation trigger: Phase gate before execution begins. Fire once per feature/epic plan, not per-ticket. Per-ticket architecture review creates noise—**the Ellipsis lesson is that dozens of micro-reviews produce duplication and developer fatigue**. (Tribe) (tribe) At the ticket level, rely on the pre-commit and security agents instead.

Required context set (minimum viable):

- Module structure via `tree` output or `GlobTool` exploration
- SQLAlchemy model definitions (`models/` directory)
- Pydantic schema definitions (`schemas/` directory)
- ADR directory contents (`docs/adr/*.md`)
- The plan document itself (epic spec or implementation blueprint)
- `CLAUDE.md` architecture rules section

The agent should **not** receive the full codebase. ZacheryGlass's production architecture reviewer (528 GitHub stars) demonstrates the correct pattern: the agent first runs `ls -R`, `glob`, and

`grep` to build its own mental model, then evaluates against principles. Give it read-only tools and let it fetch what it needs.

System prompt structure (~2,000 tokens):

```xml
<role>You are a principal software architect reviewing implementation plans for a Fa
<instructions>
Evaluate the provided plan against these architecture rules:
1. Layer separation: routes/ → services/ → repositories/ → models/
2. Services NEVER import from fastapi (no Request/Response objects in business logic
3. Repositories contain zero business logic
4. SQLAlchemy models in models/, Pydantic schemas in schemas/ — no mixing
5. No circular imports between modules
6. ADR compliance: check each relevant ADR in docs/adr/
Read the codebase structure first using Glob and Grep to understand current state be
</instructions>
<output_format>
Return findings as structured JSON:
- severity: CRITICAL | WARNING | NIT
- principle_violated: which rule from above
- location: file path and relevant section of plan
- evidence: what you observed that indicates the violation
- recommendation: specific fix with code path references
Include an overall verdict: APPROVE | REVISE | BLOCK
</output_format>
<constraints>
- Only flag issues where you can cite evidence from the plan or codebase
- If you cannot verify a concern, state it as UNCERTAIN and explain what additional
- Do not flag style issues — focus exclusively on structural and architectural conce
- Do not suggest implementation — this is review, not coding
</constraints>
```

Output contract:

```json
```

```
{
  "verdict": "REVISE",
  "findings": [
    {
      "severity": "CRITICAL",
      "principle_violated": "layer_separation",
      "location": "plan section 3.2 — OrderService",
      "evidence": "Plan specifies OrderService will import Request from fastapi to e
      "recommendation": "Extract user context in route handler, pass user_id as para
    }
  ],
  "adr_compliance": [
    {"adr": "ADR-003", "status": "COMPLIANT", "notes": "API versioning follows docum
    {"adr": "ADR-007", "status": "VIOLATION", "notes": "Plan introduces direct DB qu
  ]
}
```

Known failure mode: The agent invents coupling risks that don't exist because it infers behavior from partial import chains without seeing guard clauses in helper files. Archbot documented a case where a senior engineer spent 20 minutes disproving a confidently-stated "blocker"—the guard clause was two files away and never loaded into context. Platform Toolsmith dev **Mitigation:** The self-critique constraint ("only flag issues where you can cite evidence") combined with the UNCERTAIN escape hatch reduces this significantly.

Agent 2: Security review before auth and payment tickets

Invocation trigger: Before any ticket touching auth, payments, external APIs, or PII handling. Use file-pattern targeting in Claude Code: when the plan or ticket references `auth/`, `payments/`, or `external_apis/`, this agent fires. Can also be manually invoked via `@security-reviewer` in the prompt.

Minimum viable context set (without full codebase):

- Endpoint definition (route, HTTP method, request/response schemas)
- Authentication/authorization policy file describing which auth mechanisms exist
- Data model schema for the endpoint being touched, with PII fields annotated
- Dependency manifest (pyproject.toml or requirements.txt)
- **Static analysis output from Bandit and pip-audit—YES, include it.** The AWS `sample-mcp-security-scanner` MCP server integrates Checkov + Semgrep + Bandit github into a unified GitHub JSON output specifically designed for LLM consumption. Markaicode's production pipeline feeds Bandit output to an LLM "orchestrator" that performs context

analysis—downgrading false positives like "string concatenation in a log statement flagged as XSS" when the output goes to structured logging.

Why static analysis output matters as input: The LLM adds value that static analysis alone cannot: it understands business logic context. Bandit flags all `execute()` calls; the LLM can distinguish between a parameterized query in a repository method and a raw SQL string built from user input. Endor Labs (2025) found that traditional SAST tools miss "changes that are technically valid but shift your security architecture"—like a new unauthenticated endpoint on a destructive operation. The LLM catches architectural security changes.

Critical finding: A March 2026 study of Claude Code, OpenAI Codex, and Google Gemini building applications found all three repeat **decade-old security mistakes**. Broken access control was universal. Unauthenticated endpoints on destructive operations appeared consistently across all three agents. This is why the security review agent exists: the coding agent itself will introduce these vulnerabilities.

MCP tooling: Install `sinewaveai/agent-security-scanner-mcp` via `npx agent-security-scanner-mcp init claude-code`. This provides `scan_packages` (checks imports against 4.3M+ known packages across 7 ecosystems), `check_package` (individual verification), and `scan_security` (1,700+ vulnerability rules with AST and taint analysis). GitHub Also install `aws-samples/sample-mcp-security-scanner` for integrated Bandit + Semgrep scanning.

Anti-pattern: security review without threat model context. OpenAI's Codex Security (March 2026) builds a project-specific threat model as the first step—capturing "what the system does, what it trusts, and where it is most exposed." Without this, the security agent produces generic OWASP checklists rather than targeted findings. Include a `THREAT_MODEL.md` or equivalent in the agent's context.

Agent 3: TDD review needs both stories and conftest

Invocation trigger: Before execution of any implementation ticket. This agent validates the test plan, not the implementation. Fire it after the story/spec is written but before the developer (human or AI) begins coding.

The answer to "story files alone or conftest context" is unambiguously both, for different validation tasks:

Story files alone validate whether acceptance criteria are testable. The agent checks: Are criteria binary (pass/fail)? Are they in Given/When/Then format? Do they cover happy path, error cases, and edge cases? Does each criterion contain concrete values or thresholds that can be asserted against? "The system should be fast" fails validation. "Response time < 200ms at p95" passes.

Conftest architecture context validates whether the test infrastructure can actually support the proposed test plan. The agent checks: Do required factory_boy factories exist for all model dependencies? Are SubFactory chains complete (e.g., `BookFactory` → `AuthorFactory`)? Are

fixtures registered at the correct scope? Are there circular dependencies in the factory graph? Does the database session fixture configuration support the proposed test isolation strategy?

System prompt structure (~1,800 tokens):

```xml
<role>You are a TDD specialist validating test plans for a pytest + factory_boy + SQ
<instructions>
Validate the provided story/spec against two dimensions:

ACCEPTANCE CRITERIA TESTABILITY:
1. Each criterion must be binary (pass/fail deterministic)
2. Given/When/Then structure required
3. Must cover: happy path, at least one error case, at least one edge case
4. No ambiguous qualifiers ("user-friendly", "fast", "intuitive")
5. Concrete values, thresholds, or behaviors that map to assertions

TEST INFRASTRUCTURE READINESS:
1. Check conftest.py files for required fixture availability
2. Verify factory_boy SubFactory chains are complete
3. Confirm factory registrations via pytest-factoryboy exist
4. Validate fixture scope alignment (function-scoped for DB isolation)
5. Flag any test that depends on state from another test
</instructions>
<output_format>
{
  "ac_validation": [
    {"criterion": "AC-001", "testable": true|false, "issues": ["..."], "suggested_re
  ],
  "infrastructure_readiness": [
    {"requirement": "UserFactory", "status": "EXISTS|MISSING|INCOMPLETE", "location"
  ],
  "test_ordering": "VALID|INVALID — [explanation]",
  "verdict": "READY|NEEDS_REVISION"
}
</output_format>
```

Test ordering validation in practice: The TDAD framework (arXiv, 2026) builds a code-test dependency graph from the repository—mapping which tests should run for which code changes. (arXiv) For the TDD review agent, the simpler version is sufficient: validate that proposed tests follow unit → integration → E2E ordering, that no test depends on side effects from a previous test, and that database fixtures use function scope for isolation.

Factory dependency chain checks to implement:

- Every `SubFactory(X)` reference has a corresponding factory `X` that exists and is registered Readthedocs
- `pytest-factoryboy` has built-in cycle detection, GitHub but the agent should flag potential cycles before tests are written
- `_meta.sqlalchemy_session` configuration must align with test session fixtures Medium
- LazyAttribute/LazyFunction evaluations must not reference undefined attributes

Anti-pattern: validating test plans without the conftest. An agent that reviews only the story file will approve a test plan requiring `AdminUserFactory` when no such factory exists—then the developer wastes a full TDD cycle discovering infrastructure gaps during implementation rather than during planning.

Agent 4: Pre-commit explanation beats silent auto-fixing

Invocation trigger: Pre-commit hook or manual invocation after implementation, before commit. Runs ruff, mypy, pytest, pip-audit, and bandit as a quality gate.

The design decision is explain-don't-fix, and the evidence is clear. Microsoft's AgentRx framework Microsoft (2026) demonstrated **+23.6% improvement in failure localization accuracy** Microsoft using structured explanation over conversational LLM prompting. Developer trust in AI tools is declining (Stack Overflow 2025: 43% → 33% trust AI accuracy), diffray and auto-fixing without explanation accelerates this erosion. Developers who don't understand why a fix was applied will revert it or introduce the same pattern elsewhere.

The structured format wins over conversational. Each finding should follow: **error → root cause → options with trade-offs.** Not "here's what's wrong and here's the fix" but "here's what's wrong, here's why it's wrong at a conceptual level, and here are 2-3 ways to address it with different effort/safety trade-offs."

Concrete output format per finding:

```json
```

```
{
  "tool": "bandit",
  "rule": "B608",
  "severity": "HIGH",
  "location": {"file": "app/routes/users.py", "line": 42},
  "error": "Possible SQL injection via string-based query construction",
  "root_cause": "User input from request.query_params['id'] is interpolated directly
  "options": [
    {
      "approach": "Use SQLAlchemy text() with bound parameters",
      "trade_off": "Safest approach; requires refactoring the query builder",
      "effort": "medium",
      "example": "db.execute(text('SELECT * FROM users WHERE id = :id'), {'id': user
    },
    {
      "approach": "Use the repository's existing get_by_id() method",
      "trade_off": "Already parameterized; simplest change if method exists",
      "effort": "low"
    },
    {
      "approach": "Suppress with # nosec B608",
      "trade_off": "Only valid if this code path is provably unreachable from user i
      "effort": "trivial"
    }
  ]
}
```

Tool integration—all five tools output JSON natively:

Tool	JSON Flag	Key Output Fields
ruff	`--output-format json`	rule code, message, file, line, fix available
mypy	`--output json`	error code, message, file, line, column
pytest	`--json-report` (plugin)	test name, outcome, duration, longrepr
bandit	`-f json`	test_id, severity, confidence, issue_text, line_number
pip-audit	`--format json`	name, version, vulns (id, fix_versions, description)

Run all five tools, collect JSON outputs, feed the combined results to the agent with the source code context. The agent's job is translation and decision support, not auto-remediation.

MCP tooling: The `drewsonne-ruff-mcp-server` provides `ruff_check` and `ruff_format` tools. LobeHub The `jwilger-mcp-pytest-runner` provides `discover_tests` and `run_tests` with

structured results. LobeHub The Tweag `Pre-Commitator` tool integrates Bandit, Semgrep, and other tools specifically designed for "validating AI-generated code" with clear, AI-readable error messages. Tweag

Anti-pattern: the silent auto-fixer. GitHub Issue #108 on pre-commit-hooks captures the core problem: "Hooks that modify the contents can be dangerous and surprising... I'd prefer that none of the hooks perform any mutation and allow the user to take manual action." GitHub This is doubly true for AI-generated fixes, where the model might "fix" a type error by removing the type annotation rather than correcting the underlying logic.

Agent 5: Hallucinated package detection is a solved MCP problem

Invocation trigger: Post-implementation, after any ticket that introduces new dependencies. Also runs as part of the pre-commit quality gate.

The threat is real and quantified. The USENIX Security 2025 study tested 16 LLMs generating 576,000 code samples and found **19.7% of all recommended packages were hallucinated**—440,445 total instances, 205,474 unique fabricated names. **43% of hallucinated packages repeated consistently** across 10 re-queries, making them exploitable. DevOps Attackers register these names preemptively—a practice now called "slopsquatting" Trendmicro diffray (coined by Seth Larson, Python Security Developer-in-Residence).

The combined attack surface is worse than either dependency confusion or hallucination alone. Andrew Nesbitt Andrew Nesbitt's analysis (December 2025) explains: LLMs hallucinate consistently based on training data patterns that include leaked internal code. An attacker squats hallucinated names, and some will match real internal package names. **This inverts the discovery problem** that traditionally made dependency confusion attacks difficult. Andrew Nesbitt

MCP-compatible detection tools exist today:

The `sinewaveai/agent-security-scanner-mcp` is the most comprehensive option. Its `scan_packages` tool checks imports against **4.3M+ known packages** across 7 ecosystems (PyPI, npm, RubyGems, crates.io, pub.dev, CPAN, raku.land). GitHub npm It detects hallucinated packages like `flask-async-helpers` while confirming real packages like `flask`, `sqlalchemy`, `pydantic`. GitHub Install with `npx agent-security-scanner-mcp init claude-code`.

The `mcp-pypi` server (v3.0.0, `pip install mcp-pypi`) is security-focused: it searches packages, scans for vulnerabilities, audits dependency trees, and checks vulnerabilities before suggesting packages. It can detect transitive dependency vulnerabilities. PyPI

The `pypi-query-mcp-server` provides `get_package_versions`—critical for verifying that a package exists at a specific pinned version. GitHub A package might exist but not at the version the LLM hallucinated.

Version-specific verification is essential. The PyPI JSON API at `https://pypi.org/pypi/{package_name}/{version}/json` returns 404 if the package/version combination doesn't exist. The agent should cross-reference every pinned dependency against this endpoint.

Building a custom verification tool with FastMCP takes 20 lines:

```python
from fastmcp import FastMCP
import requests

mcp = FastMCP("DependencyVerifier")

@mcp.tool
def verify_pinned_dependency(package: str, version: str) -> dict:
    """Check if a PyPI package exists at the exact pinned version."""
    resp = requests.get(f"https://pypi.org/pypi/{package}/{version}/json")
    return {"exists": resp.status_code == 200, "package": package, "version": versio
```

Anti-pattern: relying solely on PyPI lookup for safety. RAG-based verification against live registries offers a "false sense of security" because an adversary can trivially register a new package. (Trend Micro) (USENIX) The correct layered approach is: (1) check package existence, (2) check package age (flag anything registered < 90 days before the model's training cutoff), (3) use lockfiles with hash verification (`pip install --require-hashes`), (4) run installations in isolated containers, and (5) use `GuardDog` (Datadog) for malicious package heuristic scanning.

Agent 6: Performance diagnosis maps load tests to SLO targets

Invocation trigger: After load test execution in CI/CD, before promotion to the next environment. Also invocable manually after local Locust or k6 runs.

Input format—use k6's `handleSummary()` JSON as the primary format. It provides pre-aggregated percentiles (p50, p90, p95, p99) per endpoint in a structured format that is directly LLM-friendly. For Locust, use `--csv` output which produces `_stats.csv` (per-endpoint summary), `_response_times.csv` (percentile distributions), `_failures.csv`, and `_stats_history.csv` (10-second rolling windows for trend analysis). (Locust) (LinkedIn)

SLO definitions should follow the Keptn YAML format—the industry-standard open-source implementation used by Intuit, Dynatrace customers, and CNCF adopters:

```yaml
```

```yaml
spec_version: "1.0"
comparison:
  compare_with: "single_result"
  aggregate_function: "avg"
objectives:
  - sli: response_time_p99
    pass:
      - criteria:
          - "<500"
    warning:
      - criteria:
          - "<750"
    weight: 2
  - sli: error_rate
    pass:
      - criteria:
          - "<0.1"
    weight: 1
total_score:
  pass: "90%"
  warning: "50%"
```

The output contract should include four components: SLO violation assessment, regression detection against baseline, root cause hypotheses, and a gate decision.

```json
```

```
{
  "overall_status": "FAIL",
  "score": 45,
  "slo_violations": [
    {
      "endpoint": "POST /api/orders",
      "metric": "p99_latency_ms",
      "target": 500,
      "actual": 1200,
      "deviation_pct": 140,
      "severity": "CRITICAL"
    }
  ],
  "regressions": [
    {
      "endpoint": "GET /api/users",
      "metric": "p99_latency_ms",
      "baseline": 320,
      "current": 620,
      "change_pct": 93.7,
      "severity": "WARNING"
    }
  ],
  "findings": [
    {
      "severity": "CRITICAL",
      "affected_endpoints": ["POST /api/orders"],
      "probable_causes": ["N+1 query pattern under load", "Connection pool exhaustio
      "recommended_actions": ["Profile DB queries for /api/orders", "Check pool size
      "evidence": {"latency_inflection_vus": 80, "error_correlation": 0.85}
    }
  ],
  "gate_decision": "BLOCK"
}
```

Severity classification follows the OneUptime pattern: Use p50 to detect broad regressions, p95 to tune system performance, p99 to expose architectural bottlenecks. The ratio alert pattern (**"p99 > 3 × p50 for sustained period"**) detects latency divergence indicating tail-latency problems.

MCP tooling: NaveenKumar Namachivayam's `locust-mcp-server` and `k6-mcp-server` (GitHub: `qainsights`) enable natural-language control of load testing from Claude Code. NeoLoad became the first commercial performance testing tool with native MCP support in 2025. Grafana Cloud k6 added AI-powered insights connecting test outcomes to SLOs.

Present data as markdown tables, not raw dumps. LLMs handle ~1,000–2,000 tokens of structured summary far better than raw CSV files. Neubird Pre-compute the statistical summaries, present them with SLO targets inline for comparison, and include baseline data from the previous passing run for regression detection.

Anti-pattern: threshold-only analysis. Threshold-based checks catch hard SLO violations but miss gradual regressions. Keptn's "compare with previous passing result" pattern—relative comparison across builds—catches regressions that are within SLO today but trending toward violation. The agent should use both threshold-based (for violations) and trend-based (for regressions) analysis.

The MCP stack your workflow needs

The complete MCP server configuration for the described tech stack, with all servers verified as available and actively maintained:

Integration	Server	Transport	Key Capability
Linear	`https://mcp.linear.app/mcp` (official)	HTTP	21 tools: full CRUD on issues, projects, docs
Figma	`https://mcp.figma.com/mcp` (official)	HTTP/OAuth	Design context extraction, Code Connect mapping
GitHub	`https://api.githubcopilot.com/mcp/` (official)	HTTP	PRs, issues, CI/CD status, code search, security alerts
PostgreSQL	`crystaldba/postgres-mcp`	stdio	Schema introspection, EXPLAIN plans, pg_stat_statements
Redis	`redis/mcp-redis` (official)	stdio	All data types, pub/sub, vector embeddings
Security scanning	`sinewaveai/agent-security-scanner-mcp`	stdio	4.3M+ package verification, 1,700+ vuln rules
PyPI verification	`mcp-pypi` + `pypi-query-mcp-server`	stdio	Package existence, version checking, dependency audit
Python linting	`drewsonne-ruff-mcp-server`	stdio	Ruff check, format, fix with config support
Test execution	`jwilger-mcp-pytest-runner`	stdio	Test discovery, execution, structured results
Load testing	`qainsights/k6-mcp-server`	stdio	Natural-language k6 test control

LangGraph integration uses the `langchain-mcp-adapters` package with `MultiServerMCPClient`, which connects to multiple MCP servers simultaneously across both stdio (local tools) and HTTP (remote services like Linear) transports. This enables orchestrating the full phase-gated workflow from a single LangGraph graph.

Critical security note on MCP: A February 2026 scan found 8,000+ MCP servers on the public internet, 492 with zero authentication or encryption. Three CVEs were found in Anthropic's own official Git MCP server (path traversal, argument injection, repository scoping bypass).

CyberDesserts **Use stdio transport for local tools, apply least privilege to tool permissions, validate all inputs, and run servers in sandboxed environments.** The MCP spec does not enforce audit, sandboxing, or verification—security is entirely implementation-dependent.

Conclusion: the patterns that separate production agent systems from demos

The most important insight across all six agents is that **the terminal tool pattern—forcing structured output via a submit action rather than freeform text—is the single highest-leverage design decision.** Every production system surveyed (Archbot, Ellipsis, Claude Code's built-in verification specialist, Keptn quality gates) converges on this. Freeform prose lets models sound confident about wrong findings. Structured fields with citation requirements force accountability. Neubird

The second insight is that **agent scope determines agent quality more than prompt engineering does.** Shopify learned this with 50+ tools degrading performance. shopify Ellipsis learned it by splitting one mega-agent into dozens of specialists. Tribe tribe The architecture reviewer should never see security concerns. The security reviewer should never comment on test ordering. Scope restriction via Claude Code's `tools` field (limiting to read-only for review agents) is a mechanical enforcement of this principle. Claude

The third insight concerns trust calibration. Developer trust in AI tools is declining—**43% to 33% between 2024 and 2025**—and false positives are the primary cause. diffray CodeAnt AI's multi-model consensus approach (flag only when 2+ of 3 models agree) cuts false positives ~60% while maintaining 92% recall. Codeant Ellipsis's feedback-based filtering (embedding search over past developer reactions) provides continuous improvement. tribe The explain-don't-fix pattern preserves developer agency. These are not optional polish—they determine whether the agent system survives contact with a real engineering team.

The hallucinated dependency problem is the most underappreciated risk in LLM-assisted development. Nearly one in five package suggestions is fabricated, and the slopsquatting attack surface means those fabrications are actively weaponized. Trendmicro +2 The `agent-security-scanner-mcp` tool should be considered mandatory infrastructure, not optional tooling. GitHub npm Combined with hash-pinned lockfiles and isolated installation environments, Trend Micro this closes the most dangerous gap in the workflow.